

SUPCONTENT

Documentation Technique

Réseau social de découverte cinéma & séries

Dépôt Git : <https://github.com/Miklva/3PROJ>

Juin 2025

Sommaire

1. Installation et prérequis
2. Guide de déploiement
3. Justification des choix technologiques
4. Diagrammes UML
5. Schéma de la base de données
6. Dépôt Git
7. Sécurité et gestion des secrets

Annexe - Structure du projet

1. Installation et prérequis

1.1 Environnement requis

Les outils suivants doivent être installés sur la machine hôte avant de lancer le projet.

Outil	Version minimale	Utilité
Docker	24.x	Conteneurisation des services
Docker Compose	2.x	Orchestration multi-conteneurs
Git	2.x	Récupération du code source

Pour le développement sans Docker, Node.js 18+ est également nécessaire.

1.2 Obtention de la clé API TMDB

SUPCONTENT repose sur l'API The Movie Database (TMDB) pour récupérer les métadonnées des films et séries. La clé est obligatoire au démarrage.

- Créer un compte gratuit sur <https://www.themoviedb.org/>
- Accéder à : Profil > Paramètres > API
- Remplir le formulaire de demande (usage : personnel / développement)
- Copier la valeur "Clé API (v3 auth)"

1.3 Configuration OAuth2 (optionnel)

La connexion via Google et GitHub est optionnelle. Pour l'activer, il faut créer une application OAuth2 sur chaque plateforme et récupérer un Client ID et un Client Secret.

Google : <https://console.cloud.google.com/> > Credentials > Create OAuth client ID

GitHub : <https://github.com/settings/developers> > New OAuth App

URL de callback à configurer dans les deux cas :

```
http://localhost:5000/api/auth/google/callback  
http://localhost:5000/api/auth/github/callback
```

1.4 Variables d'environnement

Copier le fichier d'exemple à la racine du projet, puis renseigner les valeurs :

```
cp .env.example .env
```

Variable	Obligatoire	Description
TMDB_API_KEY	Oui	Clé API TMDB v3
JWT_SECRET	Oui	Secret de signature des tokens JWT (chaîne longue et aléatoire)
DB_PASSWORD	Oui	Mot de passe de l'utilisateur MySQL
MYSQL_ROOT_PASSWORD	Oui	Mot de passe root MySQL
GOOGLE_CLIENT_ID	Non	ID client OAuth Google
GOOGLE_CLIENT_SECRET	Non	Secret OAuth Google
GITHUB_CLIENT_ID	Non	ID client OAuth GitHub
GITHUB_CLIENT_SECRET	Non	Secret OAuth GitHub
FRONTEND_URL	Non	URL du frontend pour les redirections OAuth (défaut : http://localhost:3000)

Important : le fichier .env ne doit jamais être committé. Il est présent dans .gitignore.

2. Guide de déploiement

2.1 Via Docker Compose (recommandé)

Le projet est entièrement conteneurisé. Un seul fichier docker-compose.yml orchestre trois services indépendants : mysql, backend et web.

- Cloner le dépôt :

```
git clone https://github.com/Miklva/3PROJ.git
cd 3PROJ
```

- Configurer les variables d'environnement :

```
cp .env.example .env # puis éditer .env avec vos valeurs
```

- Construire et lancer :

```
docker compose up --build
```

- L'application est accessible sur :

```
http://localhost:3000
```

Pour arrêter et supprimer les données (utile en développement) :

```
docker compose down -v
```

2.2 Architecture des services

Service	Image / Build	Port	Rôle
mysql	mysql:8.0	3306	Base de données relationnelle. Initialisée via schema.sql au premier démarrage.
backend	Build depuis ./server	5000	API REST Express + WebSocket Socket.io. Démarre uniquement quand MySQL est prêt (healthcheck).
web	Build depuis ./web	3000	Application React compilée, servie par Nginx en mode production.

Le backend dépend de MySQL via la directive depends_on avec condition: service_healthy. MySQL doit répondre au ping avant que le backend ne soit lancé.

2.3 Mode développement sans Docker

Pour itérer rapidement sans reconstruire les conteneurs à chaque modification :

Terminal 1 - Backend :

```
cd server
npm install
npm start
```

Terminal 2 - Frontend :

```
cd web
npm install
npm run dev
```

Le client est alors disponible sur <http://localhost:5173> (Vite dev server avec HMR).

2.4 Application mobile Android (Capacitor)

L'application web est encapsulée dans une application Android native via Capacitor. Le répertoire `web/android/` contient le projet Android pré-configuré.

- Compiler le web : `npm run build` (dans `/web`)
- Synchroniser Capacitor : `npx cap sync android`
- Ouvrir Android Studio : `npx cap open android`
- Générer l'APK depuis Android Studio (Build > Generate Signed APK ou Build APK)

En développement, l'application mobile peut pointer vers le serveur local via `capacitor.config.ts` (paramètre `server.url`).

3. Justification des choix technologiques

3.1 API tierce - The Movie Database (TMDB)

Critère	Détail
Gratuité	Gratuite pour usage non commercial avec des limites généreuses (40 requêtes / 10 secondes).
Richesse des données	Couvre films, séries, acteurs, genres, crédits, images haute résolution, bandes-annonces.
Localisation	Les réponses sont disponibles en français via le paramètre language=fr-FR.
Documentation	Documentation officielle complète, stable et maintenue activement.
Fiabilité	API mature utilisée par des milliers d'applications tierces, taux de disponibilité élevé.

Architecture de consommation : le backend agit comme proxy et cache entre les clients et TMDB. Les clients ne contactent jamais TMDB directement. Les données sont mises en cache dans la table media_cache avec une durée de vie de 7 jours, ce qui réduit les appels API redondants et améliore les temps de réponse.

3.2 Backend - Node.js / Express.js

Dépendance	Version	Rôle
Express.js	4.18	Framework HTTP minimaliste et flexible
Socket.io	4.8	WebSockets pour la messagerie temps réel
Passport.js	0.7	Abstraction OAuth2 (Google, GitHub)
mysql2	3.18	Driver MySQL performant avec support des Promises
bcryptjs	3.0	Hachage sécurisé des mots de passe (salt factor 10)
jsonwebtoken	9.0	Authentification stateless par JWT (expiration 7 jours)
express-validator	7.3	Validation des données entrantes (email, mot de passe)
multer	2.0	Upload et stockage des avatars utilisateurs
axios	1.7	Appels HTTP vers l'API TMDB
dotenv	17.3	Chargement des variables d'environnement depuis .env

Le choix de JavaScript côté serveur permet un écosystème unifié avec le client web (TypeScript), facilitant le partage de types et de logique entre les deux couches.

3.3 Client web - React / TypeScript / Vite

Dépendance	Version	Rôle
React	19	Bibliothèque UI réactive, composants réutilisables
TypeScript	5.9	Typage statique pour une meilleure maintenabilité
Vite	7	Bundler rapide avec HMR natif en développement
React Router	7	Routage côté client, navigation SPA fluide
SCSS	-	Préprocesseur CSS pour les thèmes (clair/sombre) et les variables de style
Axios	1.13	Client HTTP avec gestion centralisée des tokens JWT via intercepteurs
Socket.io-client	4.8	Communication temps réel avec le backend pour la messagerie

3.4 Client mobile - Capacitor / Android

Critère	Détail
Code partagé	Une seule base de code React couvre le web et le mobile, sans duplication de logique métier.
Cross-platform	Capacitor cible Android (et potentiellement iOS) depuis le même projet web.
Intégration	Le projet Android Studio est pré-configuré dans web/android/ avec icônes et splash screen.

3.5 Base de données - MySQL 8.0

Critère	Détail
Modèle relationnel	Adapté aux nombreuses relations du domaine : utilisateurs, critiques, listes, abonnements, messages.
Intégrité référentielle	Contraintes FOREIGN KEY avec ON DELETE CASCADE garantissant la cohérence des données.
Maturité	Solution éprouvée, excellente intégration Docker, large documentation disponible.
Support ACID	Transactions fiables pour les opérations critiques (inscription, suppressions en cascade).

4. Diagrammes UML

4.1 Diagramme de cas d'utilisation

Acteur	Cas d'utilisation
Visiteur (non connecté)	- Consulter les films et séries en tendance - Rechercher une oeuvre par titre, genre ou année - Consulter la fiche détaillée d'une oeuvre - Consulter le profil public d'un utilisateur - Consulter les listes publiques de la communauté
Utilisateur (connecté)	Tout ce que fait le visiteur, plus : - S'inscrire avec email/mot de passe (validation des champs) - Se connecter via email/mot de passe ou OAuth2 (Google, GitHub) - Gérer sa bibliothèque avec des statuts (A voir, En cours, Terminé, Abandonné) - Consulter le tableau de bord de sa collection (statistiques) - Créer des listes thématiques publiques ou privées - Ajouter, retirer et gérer des oeuvres dans ses listes - Noter une oeuvre (1 à 5 étoiles) et rédiger une critique - Modifier ou supprimer sa propre critique - Liker une critique ou y ajouter un commentaire - Signaler une critique (spoiler, insulte, contenu inapproprié) - Suivre ou ne plus suivre un autre utilisateur - Consulter son fil d'actualité (actions des personnes suivies) - Envoyer des messages privés (abonnement mutuel requis) - Gérer son profil (avatar, biographie, site web, thème, langue) - Modifier son mot de passe - Exporter ses données personnelles en JSON (conformité RGPD) - Supprimer son compte
Administrateur	Tout ce que fait l'utilisateur, plus : - Consulter les critiques signalées (avec nombre de signalements et raisons) - Supprimer une critique inappropriée - Ignorer les signalements d'une critique - Bannir ou débannir un utilisateur - Promouvoir un utilisateur au rang d'administrateur - Mettre en avant une critique (Coup de coeur)

4.2 Consultation d'une fiche média

Ce diagramme illustre le flux complet lors de l'accès à la page d'une oeuvre. Il montre le rôle de cache du backend SUPCONTENT vis-à-vis de l'API TMDB, conformément à l'exigence du cahier des charges.

Étape	De	Vers	Message / Action
1	Navigateur	Backend	GET /api/media/movie/{id}
2	Backend	MySQL	SELECT data, fetched_at FROM media_cache WHERE tmdb_id = ? AND media_type = ?
3a	MySQL	Backend	[Cache valide, age < 7 jours] Retourne le JSON en cache → passe à l'étape 7
3b	MySQL	Backend	[Cache absent ou expiré] Retourne 0 résultat → continue à l'étape 4
4	Backend	API TMDB	GET /movie/{id}?append_to_response=credits&language=fr-FR&api_key=...

Étape	De	Vers	Message / Action
5	API TMDB	Backend	JSON : métadonnées complètes (titre, synopsis, casting, genres, dates)
6	Backend	MySQL	INSERT INTO media_cache ... ON DUPLICATE KEY UPDATE (stockage du cache)
7	Backend	Navigateur	JSON : données média enrichies
8	Navigateur	Backend	GET /api/reviews/movie/{id} (en parallèle)
9	Backend	MySQL	SELECT reviews JOIN users WHERE tmdb_id = ? AND media_type = ?
10	MySQL	Backend	Liste des critiques avec moyenne des notes
11	Backend	Navigateur	JSON : critiques, moyenne, likes/commentaires
12	Navigateur	Utilisateur	Affichage de la fiche complète avec données TMDB + données communautaires

4.3 Authentification par email et mot de passe

Étape	De	Vers	Message / Action
1	Utilisateur	Navigateur	Saisie de l'email et du mot de passe dans le formulaire
2	Navigateur	Backend	POST /api/auth/login { email, password }
3	Backend	Backend	Validation des champs via express-validator
4	Backend	MySQL	SELECT id, username, password, role, is_banned FROM users WHERE email = ?
5	MySQL	Backend	Enregistrement utilisateur (ou 0 résultat si email inconnu)
6	Backend	Backend	bcrypt.compare(password, hash) - vérification du mot de passe
7a	Backend	Navigateur	[Echec] 400 { errors: [{ msg: 'Email ou mot de passe incorrect' }] }
7b	Backend	Navigateur	[Succes] 200 { token: JWT signé 7j, user: { id, username, role } }
8	Navigateur	Navigateur	Stockage du token dans localStorage + mise à jour du contexte AuthContext
9	Navigateur	Backend	Toutes les requêtes suivantes incluent : Authorization: Bearer <token>

4.4 Authentification OAuth2

Étape	De	Vers	Message / Action
1	Utilisateur	Navigateur	Clic sur 'Continuer avec Google' (ou GitHub)
2	Navigateur	Backend	GET /api/auth/google (redirect vers Google)
3	Backend	Google	Redirection vers la page de consentement Google OAuth2
4	Utilisateur	Google	Accepte les permissions demandées
5	Google	Backend	GET /api/auth/google/callback?code=... (avec le code d'autorisation)
6	Backend	MySQL	SELECT * FROM users WHERE google_id = ? OR email = ?
7a	MySQL	Backend	[Compte existant] Retourne l'utilisateur
7b	MySQL	Backend	[Nouveau compte] INSERT INTO users (username, email, google_id, avatar_url)
8	Backend	Navigateur	Redirect vers /oauth-callback?token=JWT&user=JSON_encodé
9	Navigateur	Navigateur	OAuthCallback.tsx lit les paramètres, stocke le token, redirige vers /

5. Schéma de la base de données

La base MySQL supcontent contient 10 tables. Le fichier de création est situé dans `server/src/database/schema.sql` et est exécuté automatiquement par Docker au premier démarrage du conteneur MySQL.

5.1 Vue d'ensemble des tables

Table	Rôle	Clés étrangères
users	Stocke tous les utilisateurs, locaux et OAuth	-
subscriptions	Abonnements entre utilisateurs (follow)	follower_id → users, following_id → users
reviews	Notes (1-5 étoiles) et critiques textuelles	user_id → users - UNIQUE (user_id, tmdb_id, media_type)
review_likes	J'aime sur une critique	review_id → reviews, user_id → users
review_comments	Commentaires sous une critique	review_id → reviews, user_id → users
reports	Signalements de critiques inappropriées	review_id → reviews, reporter_id → users - UNIQUE (review_id, reporter_id)
lists	Listes d'oeuvres (4 par défaut + personnalisées)	user_id → users
list_items	Oeuvres TMDB ajoutées à une liste	list_id → lists - UNIQUE (list_id, tmdb_id, media_type)
media_cache	Cache local des données TMDB (TTL 7 jours)	- (clé primaire composite : tmdb_id + media_type)
messages	Messages privés entre utilisateurs	sender_id → users, receiver_id → users

5.2 Détail des colonnes - table users

Colonne	Type	Contrainte	Description
id	INT	AUTO_INCREMENT PK	Identifiant unique
username	VARCHAR(100)	UNIQUE NOT NULL	Nom d'utilisateur
email	VARCHAR(150)	UNIQUE NOT NULL	Adresse email
password	VARCHAR(255)	NULL	Hash bcrypt - NULL pour les comptes OAuth
google_id	VARCHAR(255)	UNIQUE NULL	Identifiant Google OAuth2
github_id	VARCHAR(255)	UNIQUE NULL	Identifiant GitHub OAuth2

Colonne	Type	Contrainte	Description
bio	TEXT	NULL	Biographie libre
website_url	VARCHAR(255)	NULL	Lien vers le site personnel
avatar_url	VARCHAR(500)	NULL	URL de l'avatar (upload ou OAuth)
theme	VARCHAR(20)	DEFAULT 'light'	Préférence de thème : light ou dark
language	VARCHAR(10)	DEFAULT 'fr'	Langue de l'interface : fr ou en
role	ENUM	DEFAULT 'user'	Rôle : user ou admin
is_banned	BOOLEAN	DEFAULT FALSE	Compte banni ou non
created_at	TIMESTAMP	DEFAULT NOW()	Date d'inscription

5.3 Détail des colonnes - table media_cache

Colonne	Type	Description
tmdb_id	INT	Identifiant TMDB (partie 1 de la clé primaire composite)
media_type	ENUM('movie','tv')	Type de média (partie 2 de la clé primaire composite)
data	LONGTEXT	JSON complet retourné par TMDB, incluant les crédits (append_to_response=credits)
fetched_at	TIMESTAMP	Date du dernier appel TMDB. Mis à jour automatiquement (ON UPDATE CURRENT_TIMESTAMP).

5.4 Règles métier et contraintes

- Un utilisateur ne peut rédiger qu'une seule critique par oeuvre. La contrainte UNIQUE (user_id, tmdb_id, media_type) sur la table reviews l'impose au niveau base de données. Une deuxième soumission met à jour la critique existante (ON DUPLICATE KEY UPDATE).
- 4 listes par défaut sont créées automatiquement à la première connexion de chaque utilisateur : À voir, En cours, Terminé, Abandonné. Ces listes ne peuvent ni être renommées ni supprimées (flag is_default = TRUE).
- Un utilisateur ne peut pas signaler sa propre critique (vérification métier dans la route /api/reviews/:id/report).
- La messagerie privée est restreinte aux abonnements mutuels : les deux utilisateurs doivent se suivre réciproquement pour pouvoir échanger (vérification via la fonction isMutualFollow).
- Les titres et posters des oeuvres sont dénormalisés dans list_items pour éviter un appel API ou une jointure à chaque affichage d'une liste.

- Toutes les suppressions sont en CASCADE depuis users : supprimer un compte supprime toutes ses données associées (critiques, listes, likes, commentaires, abonnements, messages).

5.5 Index de performance

Index	Table	Colonnes	Utilité
PRIMARY KEY	media_cache	(tmdb_id, media_type)	Accès direct au cache d'une oeuvre
UNIQUE	reviews	(user_id, tmdb_id, media_type)	Une critique par utilisateur par oeuvre
idx_messages_conversation	messages	(sender_id, receiver_id)	Récupération rapide d'une conversation
idx_messages_receiver	messages	(receiver_id, is_read)	Comptage des messages non lus
UNIQUE	subscriptions	(follower_id, following_id)	Prévention des doublons d'abonnement

6. Dépôt Git

L'intégralité du code source est versionné sur GitHub. Le dépôt est privé pendant la période de rendu, conformément aux consignes du projet.

Élément	Valeur
URL du dépôt	https://github.com/Miklva/3PROJ
Branche principale	main
Visibilité	Privé jusqu'à la date de rendu, puis public

6.1 Structure des commits

Les commits suivent une convention de nommage cohérente pour faciliter la lecture de l'historique :

- feat: ajout d'une nouvelle fonctionnalité
- fix: correction d'un bug
- refactor: amélioration du code sans changement de comportement
- docs: mise à jour de la documentation
- chore: tâches de maintenance (configuration, dépendances)

6.2 Fichiers exclus du versionnement (.gitignore)

Fichier / Dossier	Raison
.env	Contient les secrets (clés API, mots de passe). Ne doit jamais être committé.
node_modules/	Dépendances installables via npm install. Trop volumineux pour Git.
uploads/	Avatars uploadés par les utilisateurs. Données de runtime, pas du code.
dist/	Build compilé du frontend. Généré automatiquement par Vite.

7. Sécurité et gestion des secrets

7.1 Principes appliqués

- Aucun secret (clé API, mot de passe, secret JWT) ne doit être présent en clair dans le code source ou dans Git. Tous les secrets transitent via des variables d'environnement chargées depuis le fichier `.env` (non versionné).
- Le fichier `.env.example` fournit la liste des variables à configurer avec des valeurs fictives, sans aucune valeur réelle.
- Les mots de passe utilisateurs sont systématiquement hachés avec `bcrypt` (salt factor 10) avant stockage. Aucun mot de passe en clair n'est jamais conservé en base.
- L'authentification repose sur des tokens JWT signés (algorithme HS256) avec une expiration de 7 jours. Le secret de signature est lu depuis la variable d'environnement `JWT_SECRET`.
- Les routes protégées vérifient le token via le middleware `auth.js` avant toute opération. Les routes d'administration vérifient en plus le rôle `admin` via `admin.js`.
- Les requêtes SQL utilisent exclusivement des requêtes paramétrées (placeholders `?`) pour prévenir les injections SQL.

7.2 Configuration avant déploiement

Avant tout déploiement (y compris en local pour le rendu), vérifier les points suivants :

Point de contrôle	Action requise
<code>JWT_SECRET</code>	Utiliser une chaîne aléatoire d'au moins 32 caractères. Ne pas utiliser la valeur d'exemple.
<code>DB_PASSWORD</code> / <code>MYSQL_ROOT_PASSWORD</code>	Utiliser des mots de passe forts, différents de ceux de l'exemple.
<code>TMDB_API_KEY</code>	Ne jamais committer la clé réelle. La stocker uniquement dans <code>.env</code> .
<code>docker-compose.yml</code>	Ne pas indiquer de valeurs de secrets en dur. Utiliser des références aux variables d'environnement.
<code>.env</code> dans <code>.gitignore</code>	Vérifier que le fichier <code>.env</code> n'apparaît pas dans <code>git status</code> avant chaque commit.

Annexe - Structure du projet

Chemin	Description
docker-compose.yml	Orchestration des 3 services (mysql, backend, web)
.env.example	Modèle des variables d'environnement à configurer
server/app.js	Point d'entrée du serveur Express + initialisation Socket.io
server/src/config/database.js	Pool de connexions MySQL2 (Promise-based)
server/src/config/passport.js	Stratégies OAuth2 Google et GitHub
server/src/middleware/auth.js	Vérification du token JWT sur les routes protégées
server/src/middleware/admin.js	Restriction d'accès aux administrateurs
server/src/routes/auth.js	Inscription, connexion email/mdp, callbacks OAuth2
server/src/routes/media.js	Proxy TMDB avec système de cache MySQL (TTL 7j)
server/src/routes/reviews.js	Critiques, notes, likes, commentaires, signalements, modération
server/src/routes/lists.js	Listes par défaut et personnalisées, list_items
server/src/routes/feed.js	Fil d'actualité agrégé (critiques + ajouts en liste)
server/src/routes/messages.js	Messagerie privée (conversations, envoi, marquage lu)
server/src/routes/users.js	Profils, follow, export RGPD, upload avatar, suppression compte
server/src/routes/admin.js	Rapports, bannissement, promotion, mise en avant
server/src/database/schema.sql	Schéma complet de la base de données (10 tables)
web/src/App.tsx	Routeur principal de l'application React
web/src/context/AuthContext.tsx	Contexte global d'authentification (token, user, login/logout)
web/src/pages/	Pages : Home, Search, MediaDetail, Profile, PublicProfile, Feed, Messages, Dashboard, Admin, Settings...
web/src/components/	Composants réutilisables : MediaCard, ReviewInteractions, Layout, Button...
web/src/translations.ts	Fichier de traductions (fr/en) pour l'internationalisation
web/src/socket.tsx	Instance partagée du client Socket.io
web/capacitor.config.ts	Configuration Capacitor (appld, webDir, serveur)
web/android/	Projet Android Studio généré par Capacitor
web/nginx.conf	Configuration Nginx pour servir le build React en production